Project 5:- Maven Selenium Project – MyMavenSeleniumApp03

Objective

This project focuses on using Selenium WebDriver within a Maven setup to automate the login process on the SauceDemo website, entering valid credentials and verifying successful authentication.

Sample Website Used for Automation

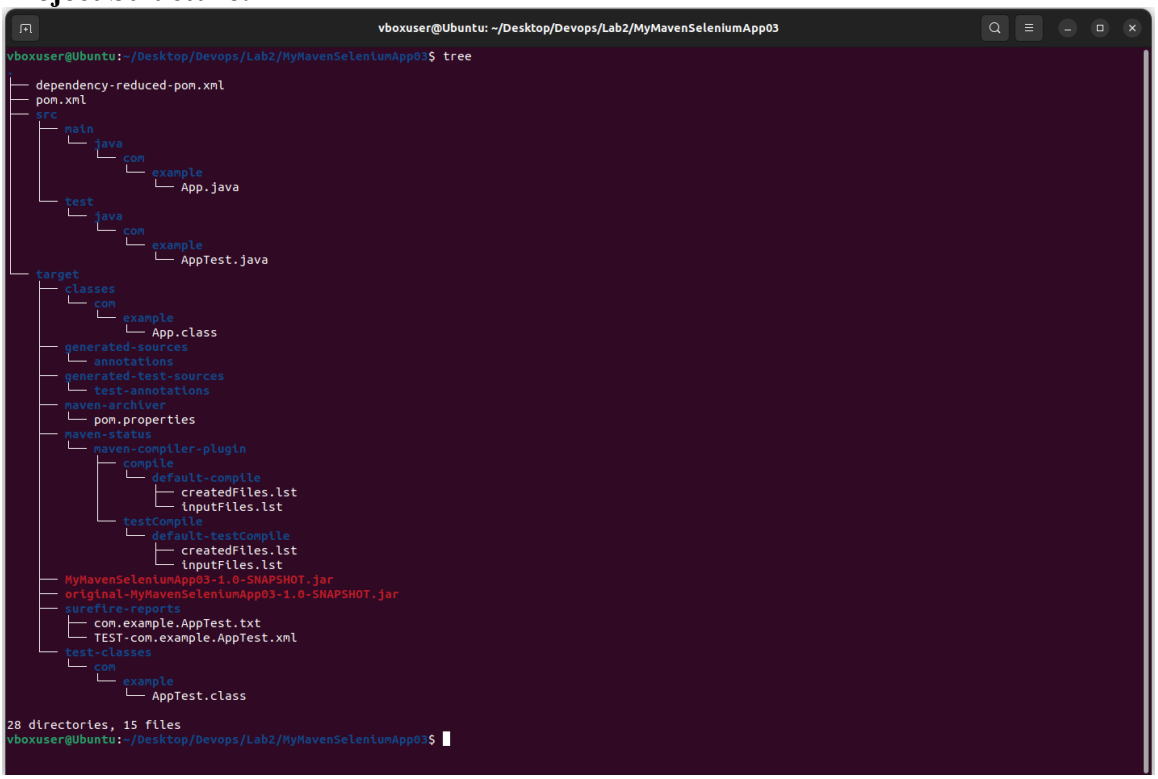
https://www.saucedemo.com/

Step 1: Create a Maven Project Using Terminal

Enter the command below in your terminal to create the Maven project:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp03
  -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

Project Structure:



Step 2: Add Selenium Dependency in pom.xml

Open pom.xml and place the following dependency inside the <dependencies> block:

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.41.0</version>
</dependency>
```

Step 3: Inspect Elements in the Website

Open DevTools in the browser and inspect the login form elements. The attributes user-name, password, and login-button are used to uniquely identify each field and are referenced in the script.

Step 4: Create App.java

Create or open App.java at src/main/java/com/example/ and enter the code shown below:

```
package com.example;


import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;


public class App {
  public static void main(String[] args) throws InterruptedException {
    WebDriver driver = new ChromeDriver();
    driver.get("https://www.saucedemo.com/");
    Thread.sleep(3000);

    driver.findElement(By.id("user-name")).sendKeys("standard_user");
    Thread.sleep(3000);
    driver.findElement(By.id("password")).sendKeys("secret_sauce");

    Thread.sleep(3000);
    driver.findElement(By.id("login-button")).click();
    Thread.sleep(3000);

    driver.quit();
  }
}
```

Step 5: Build the Maven Project

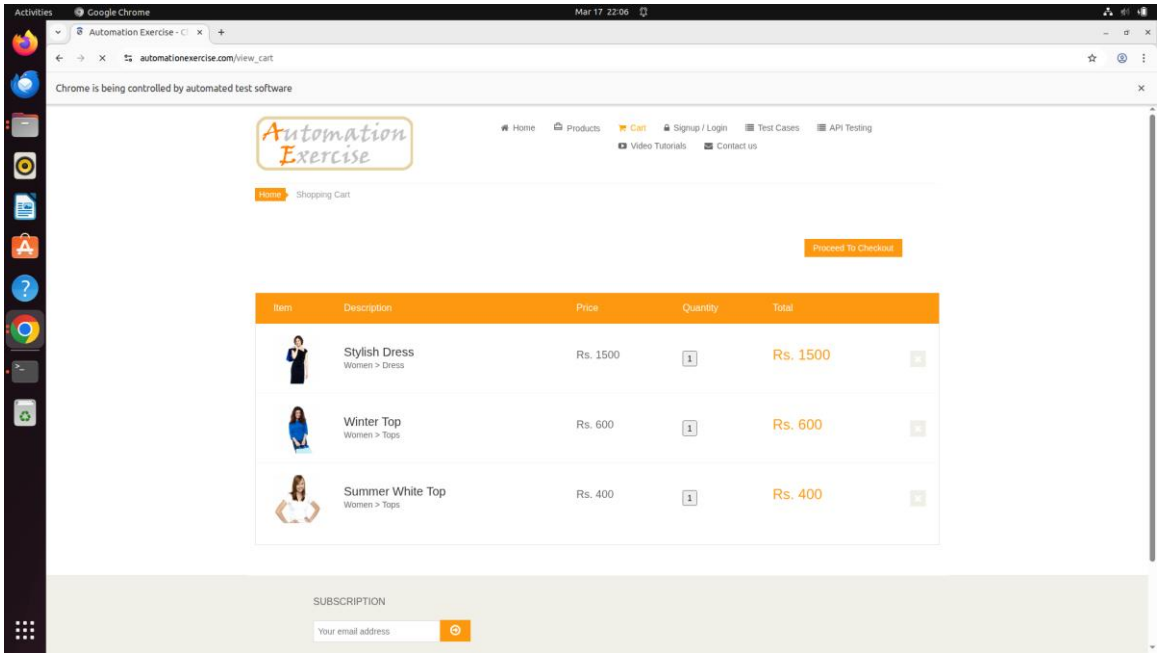
```
mvn clean compile
```

Step 6: Run the Application

```
mvn exec:java
```

Output

All three products (IDs 4, 5, and 6) are added to the cart and the cart page correctly reflects the selections.



Conclusion

This project shows that Selenium can reliably handle real-world e-commerce flows like product selection and cart management. The entire process ran automatically from start to finish.

Jenkins Pipeline for MyMavenSeleniumApp03

Step 1: Open the Jenkins Dashboard in order to create a new item.

Step 2: Click on New Item in order to create a New Item. Give the name as "MyMavenSeleniumApp03" and Item type as Pipeline, then click Create.

Step 3: Configure the settings by defining the Definition as "Pipeline Script from SCM" and SCM as "Git". Paste the Repository URL of the project from GitHub and choose Credentials as "GitHub Credentials" which was created earlier.

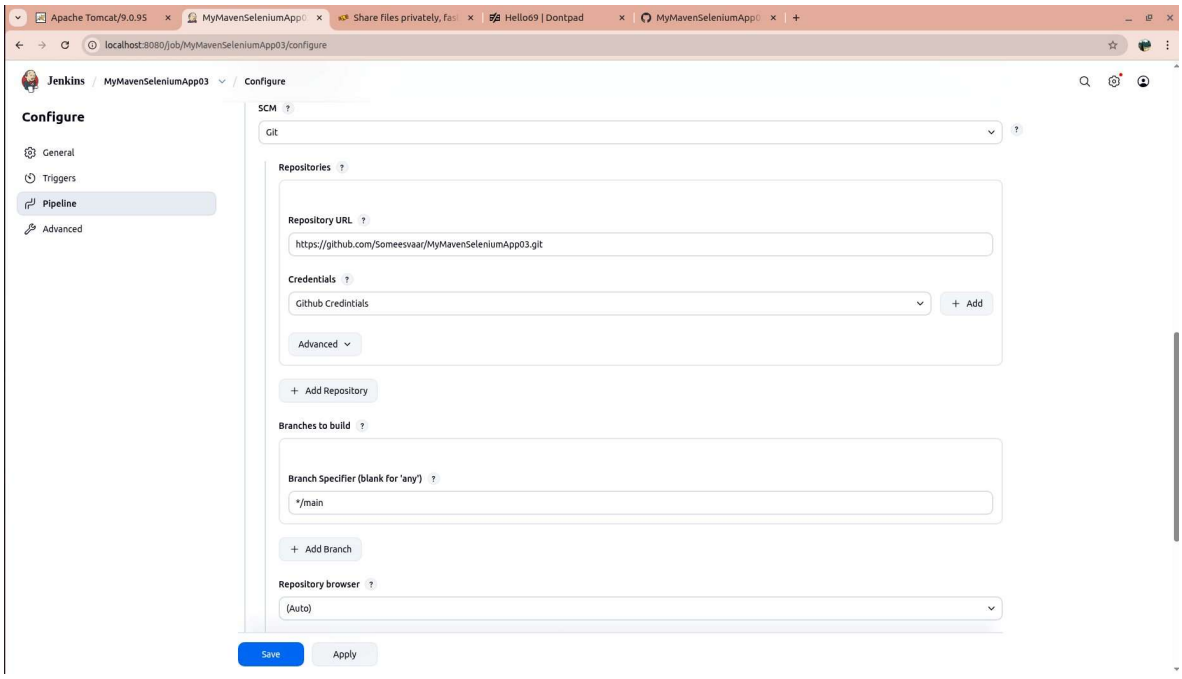


Fig C-1: Jenkins Pipeline configuration – Repository URL for MyMavenSeleniumApp03.git with GitHub Credentials

Step 4: In Terminal, go to the directory created for MyMavenSeleniumApp03 and create a new file called "Jenkinsfile".

Step 5: Paste the following code inside the Jenkinsfile:

```
pipeline {
    agent any  // Use any available agent

    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
        jdk 'JDK'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url:
'https://github.com/Someesvaar/MyMavenSeleniumApp03.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package'  // Run Maven build
            }
        }
```

```
        stage('Test') {
            steps {
                sh 'mvn test'  // Run unit tests
            }
        }




        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenSeleniumApp03-1.0-SNAPSHOT.jar'
            }
        }



    }
     post {
        success {
            echo 'Build and deployment successful!'
        }
        failure {
            echo 'Build failed!'
      }

}
```

And add the following to App.java
options.addArguments("--headless=new"); // modern headless mode
options.addArguments("--no-sandbox"); // required in Jenkins/Linux
options.addArguments("--disable-dev-shm-usage"); // prevents crashes
options.addArguments("--disable-gpu");

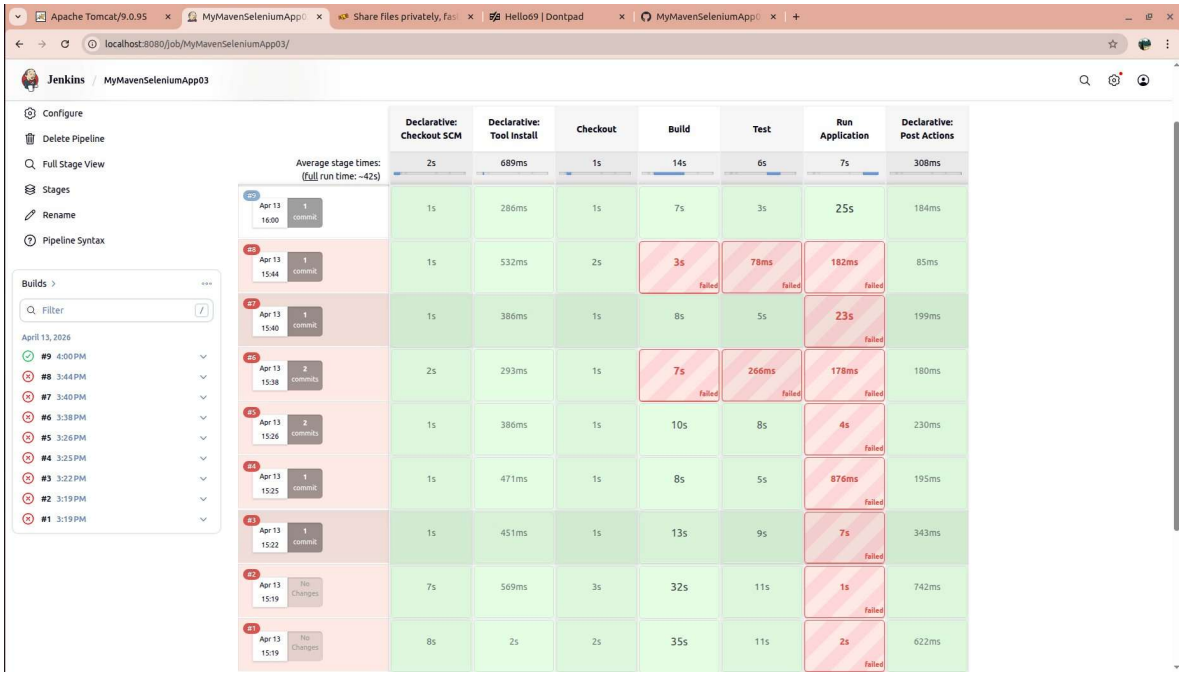


Fig C-2: Jenkinsfile committed to the MyMavenSeleniumApp03 GitHub repository

Step 6: Update pom.xml to change the versions of source and target from 1.7 to 1.8.

Step 7: Push the changes and commit by pushing the "Jenkinsfile" to the respective repository on GitHub.

Step 8: Go to Jenkins and Build the Pipeline, then check the Stage View to confirm successful execution.

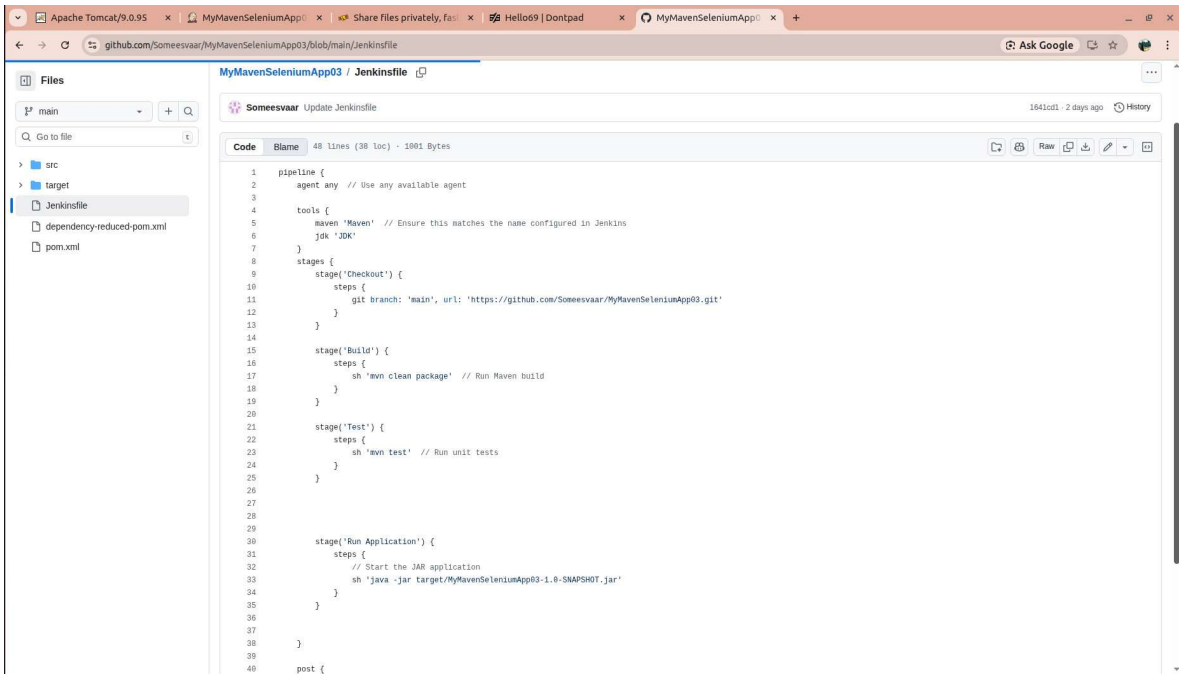


Fig C-3: Jenkins Stage View showing all 9 builds for MyMavenSeleniumApp03 (Build #9 successful, ~42s total)